Pruebas unitarias: manuales vs. generadas con IA

Kata **Tres en Raya**: misma implementación, dos suites de pruebas comparadas línea por línea.

AUTOR

Axl Severich

METODOLOGÍA

JUnit 4.13 · Maven · Red-Green-Refactor

ENTREGABLES

Repo GitHub público · prompts/ · este PDF

La kata: Tres en Raya

REQUISITOS Y PRUEBAS DEL ENUNCIADO

REQUERIMIENTO 1

Una pieza puede estar colocada en un espacio de un tablero 3×3.

- Fuera del eje X → se lanza una excepción
- Fuera del eje Y → se lanza una excepción
- Lugar ya ocupado → se lanza una excepción

REQUERIMIENTO 2

Debe existir una forma de determinar qué jugador juega después.

- El primer turno siempre lo juega `'X'`
- Si el último turno fue `'X'`, el siguiente es `'+'`
- Si el último turno fue `'+'`, el siguiente es `'X'`

REQUERIMIENTO 3

Un jugador gana si es el primero en conectar una línea de un lado a otro, o de una esquina a otra.

- Si no se cumple la condición de victoria → no hay ganador
- Gana con toda una línea horizontal ocupada
- Gana con toda una línea vertical ocupada
- Gana con toda una línea diagonal ocupada

Fuente: presentación del curso *"3. TDD - CI CD.pdf"*, páginas 16 a 18. La implementación `TresEnRaya.java` (80 líneas) y las dos suites viven en el repositorio local `TDD/`.

Cómo se comparó

MISMO CÓDIGO, DOS SUITES

Implementación única
TresEnRaya.java · 80 líneas · sin tocar

→

Suite A — Humano
TresEnRayaTest.java · 10 pruebas · construida con el ciclo TDD

Suite B — IA
TresEnRayaIATest.java · 27 pruebas · salida del prompt

→

mvn -B clean test
JUnit 4.13 + GitHub Actions (JDK 17)

SUITE A · ESCRITA POR EL ESTUDIANTE (TDD)

10 PRUEBAS @TEST	**157** LÍNEAS DE CÓDIGO
13 ASERCIONES	**10** SYSTEM.OUT.PRINTLN

SUITE B · GENERADA CON IA (PROMPT 01 + 02)

27 PRUEBAS @TEST	**251** LÍNEAS DE CÓDIGO
44 ASERCIONES	**0** SYSTEM.OUT.PRINTLN

Resultado de la suite completa: Tests run: 42, Failures: 0, Errors: 0, Skipped: 0

10 humanas + 27 IA + 5 de la kata de la Calculadora

El prompt usado con la IA

QUÉ CONTROLA EL PROMPT

1. **Rollo y contexto**
   Experto senior en TDD; proyecto Maven, Java 8, JUnit 4.13, package `com.ucb.tdd`.

2. **API y contrato exacto**
   `jugar(int x, int y)`, `siguienteTurno()`, mensajes literales de las excepciones y de los resultados.

3. **Especificación como única fuente de verdad**
   Los 3 requerimientos y sus 10 casos, copiados de la presentación del curso.

4. **Cobertura mínima exigida**
   7 bloques: límites de coordenadas, mensajes, turno, 8 líneas ganadoras, tablero lleno…

5. **Restricciones de estilo**
   `@Before`, `assertThrows`, nombres en español, sin `System.out`, sin librerías extra.

6. **Prohibición de inventar requisitos**
   Si la especificación no lo define, no se prueba. + criterio de aceptación: `mvn -B clean test` en verde.

FRAGMENTO LITERAL DEL PROMPT

```
## Cobertura mínima exigida
1. Coordenadas inválidas en X (0, 4 y negativa) y en
   Y (0, 4 y negativa), verificando también el
   mensaje exacto de la excepción.
2. Coordenadas válidas en los extremos (1,1) y (3,3)
   -> no lanzan excepción.
3. Posición ocupada -> excepción, con su mensaje.
4. Orden de turnos: primer turno 'X', alternancia
   X -> + -> X durante una secuencia.
5. Sin ganador -> "No hay ganador".
6. Las 8 líneas ganadoras (3 filas + 3 columnas +
   2 diagonales), probando que gana 'X' y que gana '+'.
7. Tablero lleno sin tres en raya -> "No hay ganador".
```

PROMPT-02 (segunda ronda): pedir a la IA una matriz de cobertura, cerrar los huecos con pruebas nuevas y **descartar los requisitos que ella misma inventó**. Salida final: `TresEnRayaIATest.java` con 27 pruebas.

Tabla comparativa

MÉTRICAS Y COMPORTAMIENTO DE CADA SUITE

CRITERIO	HUMANO · TRESENRAYATEST	IA · TRESENRAYAIATEST
Pruebas / líneas / aserciones	10 / 157 / 13	27 / 251 / 44
Cómo se generaron	Ciclo TDD (rojo-verde-refactor), una a una	De una sola pasada con el prompt + auditoría
Preparación del tablero	`new TresEnRaya()` repetido 10 veces	Un único `@Before prepararTablero()`
Código muerto	**Sí** — método `before()` sin `@Before`	**No**
Salida a consola en las pruebas	**10 println** (ruido en CI)	**0 println**
Cómo se prueban las excepciones	`@Rule ExpectedException` + `expect(Exception.class)`	`assertThrows` + `assertEquals(mensaje)`
Mensajes exactos de excepción	0 de 2 validados	2 de 2 validados (5 pruebas)
Coordenadas fuera de rango probadas	solo 5	`0`, 4, 5 y `-1` en X e Y
Líneas ganadoras cubiertas	4 de 8	8 de 8
Formas de ganar por jugador	2 de 4 para 'X', 2 de 4 para '+'	4 de 4 para 'X' y 4 de 4 para '+'
Tablero lleno sin ganador (empate)	No	Sí (1 prueba)
Casos de la especificación cubiertos	10 de 10	10 de 10
Requisitos fuera del enunciado	No plantea ninguno	Los propone y los **descarta** en el prompt 02

Diferencia 1 · Preparación y estructura

DUPLICACIÓN VS. SETUP DECLARATIVO

HUMANO — TRESENRAYATEST.JAVA

```java
@Test
public void cuandoFueraXlanzarExcepcion() throws Exception{
    tresEnRaya = new TresEnRaya();
    System.out.println("Inicializando Prueba.... " + ...);
    exception.expect(Exception.class);
    tresEnRaya.jugar(5,2);
}

// este método nunca se ejecuta:
public final void before(){ }
```

- El objeto se reconstruye dentro de **cada** prueba (×10), con el mismo println de arranque.
- `before()` **no tiene `@Before`**: el "arranque" escrito existe pero no corre nunca.
- Patrón repetido a mano "1. Preparación / 2. Lógica / 3. Verificación" en los 10 métodos.

IA — TRESENRAYAIATEST.JAVA

```java
private TresEnRaya juego;

@Before
public void prepararTablero() {
    juego = new TresEnRaya();
}

@Test
public void cuandoLaCoordenadaXEsMenorQueUnoSeLanzaExcepcion() {
    Exception e = assertThrows(Exception.class,
        () -> juego.jugar(0, 2));
    assertEquals("La coordenada 0 esta fuera de lugar",
        e.getMessage());
}
```

- Un solo punto de preparación: si cambia la API del juego, se edita **un** método.
- Cero salida a consola: el log de CI muestra solo fallos reales.
- Nombres largos y descriptivos, en un patrón consistente en los 27 métodos.

Diferencia 2 · Manejo de excepciones

"FALLE POR LA RAZÓN CORRECTA"

HUMANO — CUALQUIER EXCEPCIÓN SIRVE

```
@Rule
public ExpectedException exception = ExpectedException.none();

@Test
public void cuandoLugarOcupadoLanzarExcepcion() throws Exception {
    tresEnRaya = new TresEnRaya();
    tresEnRaya.jugar(1,3);
    exception.expect(Exception.class);  // ¡cualquiera!
    tresEnRaya.jugar(1,3);
}
```

- `ExpectedException` está **deprecado** desde JUnit 4.13.
- `Exception.class` acepta **cualquier** error: un `NullPointerException`, un fallo de tipeo o una excepción por otra causa dejaría la prueba en verde.
- Los mensajes no se verifican: 0 de 2.
- Las coordenadas 0, 4 y negativas nunca se prueban (solo 5).

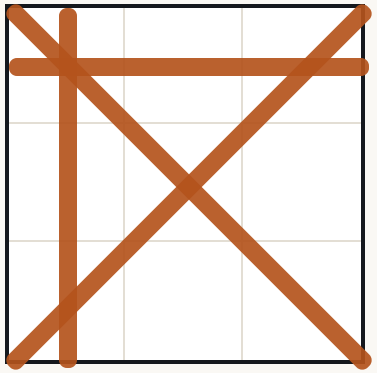
IA — SE VALIDA EL CONTRATO PÚBLICO

```
@Test
public void cuandoLaPosicionYaEstaOcupadaSeLanzaExcepcion() throws Exception {
    juego.jugar(1, 3);
    Exception e = assertThrows(Exception.class,
        () -> juego.jugar(1, 3));
    assertEquals("La posicion ya esta ocupada",
        e.getMessage());
}
```

- `assertThrows` **devuelve** la excepción: se comprueba el mensaje literal del contrato.
- Una prueba falla si llega una excepción distinta de la esperada.
- 8 pruebas de coordenadas inválidas (0, 4, 5 y −1 en ambos ejes) + 2 de extremos válidos (1,1) y (3,3).
- Los 2 mensajes del contrato: **2 de 2** cubiertos.

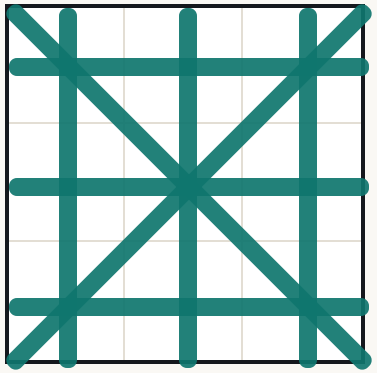
> **Riesgo sustancial:** con `expect(Exception.class)` una prueba puede quedar verde *por la razón equivocada* y ocultar un defecto real; con `assertThrows` + mensaje, la prueba verifica además que la API se llama exactamente como fue diseñada.

Diferencia 3 · Cobertura



Humano: 4 de 8

fila 1 · columna 1 · 2 diagonales



IA: 8 de 8

3 filas + 3 columnas + 2 diagonales

> La IA agrega una prueba por cada línea del tablero **y** deja que cada jugador (`'X'` y `'+'`) demuestre sus 4 formas de ganar; el humano deja sin cubrir la fila 3, la fila 2, la columna 2 y la columna 3.

CASO	HUMANO	IA
Los 10 casos de la especificación	**10/10**	**10/10**
Excepción en eje X fuera de rango	**✓ (5)**	**✓ (0, 4, 5)**
Excepción en eje Y fuera de rango	**✓ (5)**	**✓ (0, 4, 5)**
Coordenada negativa (−1)	✗	✓
Extremos válidos (1,1) y (3,3)	implícitos	**✓ explícitos**
Mensaje literal de cada excepción	**0/2**	**2/2**
Líneas ganadoras	**4/8**	**8/8**
'X' y '+' probados en las 4 formas de ganar	**2 y 2**	**4 y 4**
Turno inicial y alternancia en secuencia	parcial	✓
Tablero lleno sin ganador	✗	✓
Prueba de regresión del mensaje “No hay ganador”	✓	✓

Diferencia 4 · Alcance y proceso

HUMANO: INTENCIÓN Y PROCESO TDD

- Las pruebas nacen **antes** del código: el historial muestra el ciclo ("hacer fallar intencionalmente", "corregir el error", "rompiendo una prueba / error de regresión").
- Cada prueba toma una **decisión de diseño**: define cómo se quiere usar `jugar()` y `siguienteTurno()`.
- Sabe qué es relevante para el dominio y no prueba ruido.
- **Límite:** se detiene en el mínimo del enunciado: 4 de 8 líneas, sin límites de coordenadas ni mensajes, y con un método de setup que nunca corre.

IA: AMPLITUD Y CONSISTENCIA

- 27 pruebas en una pasada: casos límite, ambas coordenadas, los 8 tableros ganadores y el empate.
- Patrón uniforme de nombres, aserciones y comentarios en todo el archivo.
- **Riesgo controlado:** tiende a *inventar* requisitos. Ejemplos que se le pidieron y se **descartaron** en el prompt 02:

  · "jugar después de un ganador debe lanzar excepción" → no está en el enunciado y la implementación lo permite.
  · "regla propia de empate / contador de turnos" → fuera del requerimiento 3.

- No vive el ciclo rojo-verde-refactor: entrega todo de golpe, sin la disciplina de paso pequeño.

Lo que comparten: ambas suites se acoplan a los strings exactos de retorno (`"X es el ganador"`, `"No hay ganador"`), así que un cambio de formato rompe las pruebas; y la propia implementación imprime `"Ocupando posicion: x, y"` en cada jugada, ruido que convendría eliminar antes del pipeline de CI.

Fortalezas, debilidades y veredicto

BALANCE DE LA COMPARACIÓN

HUMANO · 10 PRUEBAS

FORTALEZAS

- Trazabilidad total al ciclo TDD y a la decisión de diseño.
- Cubre el 100 % de los casos pedidos en la presentación.
- Código corto, legible y con comentarios de jugada.

DEBILIDADES

- Setup duplicado + 10 `println` + `before()` muerto.
- Excepciones genéricas: posibles **falsos verdes**.
- 4 de 8 líneas ganadoras; sin límites 0/4 ni mensajes.

IA · 27 PRUEBAS

FORTALEZAS

- Cobertura 8/8 líneas, ambos jugadores, casos límite y empate.
- Verifica el contrato (mensajes exactos) con `assertThrows`.
- Consistencia, cero ruido en consola, listo para CI.

DEBILIDADES

- Puede inventar requisitos que nadie definió (filtra el prompt 02).
- No aporta el proceso TDD: no es “red-green-refactor”.
- Código más largo (251 líneas) y repetitivo en las secuencias.

Veredicto: la IA **no reemplaza** las pruebas escritas en TDD; las **amplía**. El enfoque usado aquí fue: escribir las pruebas con el ciclo rojo-verde-refactor, usar el prompt para generar una segunda suite más exhaustiva, y dejar ambas en verde con `mvn -B clean test` (42 pruebas, 0 fallos) dentro del pipeline de GitHub Actions.

Conclusiones y entregables

QUÉ APRENDIMOS

1. **La calidad del prompt define la calidad de las pruebas**
   API + contrato + especificación + cobertura mínima + estilo = una suite que compila y pasa a la primera.

2. **Sin la cláusula “no inventes requisitos”, la IA se pasa**
   Propone casos que el enunciado no pide; probarlos acopla la prueba a decisiones que nadie tomó.

3. **La IA brilla donde el humano se ahorra trabajo**
   Límites, matrices de cobertura y mensajes exactos: menos duplicación y menos “falsos verdes”.

4. **El humano sigue siendo el dueño del diseño**
   El ciclo TDD define el API; la IA solo lo verifica con más casos.

ENTREGABLES DE LA ACTIVIDAD

- **Repositorio GitHub público** (código + pruebas + workflow de CI) → `github.com/SeverichAxl/TDD_GDC`
- **Prompt(s):** `prompts/PROMPT-01-generar-pruebas-unitarias.md` y `prompts/PROMPT-02-auditoria-cobertura.md`
- **Este PDF** con el detalle de la comparación

PIPELINE DE CI (GITHUB ACTIONS)

```
# .github/workflows/maven.yml
on: push / pull_request → rama main
jobs: run-unit-tests (ubuntu-latest, JDK 17 temurin)
  - uses: actions/checkout@v4
  - uses: actions/setup-java@v4
  - run: mvn -B clean compile test --file pom.xml
```

42	0	3
PRUEBAS	FALLOS	SUITES